LifeSync

Program Documentation

A source-audited technical and user manual for the desktop wellness, nutrition, fitness and planning application.

Java 21 / JavaFX 21
Local JSON persistence
Optional Gemini AI
Light and dark themes



Edition: 1.0 - implemented-system manual
Date: 24 October 2026
Verification: 75 automated tests; zero failures and errors
Scope decision: Google Calendar integration is excluded.

Prepared from the current repository and the two supplied proposal screenshots. Illustrations use synthetic demonstration accounts. No API key, recovery code, password hash, or real account record is reproduced.

Contents

1. Document control and scope 3

2. Purpose, users and requirements 4

3. Java requirement traceability 5

4. Software Development Life Cycle evidence 7

5. System architecture and design 8

6. Data model and storage dictionary 9

7. Algorithms and pseudocode 14

8. Flowcharts and process diagrams 19

9. User interface and operating guide 23

10. Validation, exceptions and security boundaries 27

11. Verification and test evidence 28

12. Installation, execution and deployment 30

13. Operations, maintenance and troubleshooting 31

14. Limitations, glossary and source index 32

The clickable contents list the chapters. PDF bookmarks provide direct access to individual subsections.

1. Document control and scope

Item	Recorded value
Document owner	LifeSync project maintainers; no individual authorship or institutional approval is assumed.
Application version	Maven artifact com.LifeSync:LifeSync:1.0-SNAPSHOT
Reference inputs	Unknown.jpeg: Java programming requirements; Unknown-2.jpeg: SDLC stages.
Implementation baseline	Current src/, pom.xml, Maven test reports, and inspected JavaFX workflows.
Source snapshot	91352cfd9037d3ce76dbfa5105e44cab7435de59c9d3f60d1d53ddcaacea2834
Excluded integration	Google Calendar: no OAuth, calendar sync, external event writes, or calendar reminders.

This manual describes the implemented application, its development evidence and its operating procedures. It distinguishes an implemented service from a visible UI control and distinguishes automated test results from desktop inspection. The proposal class names are conceptual examples, not an instruction to create duplicate entity classes.

1.1 Changes made after inspecting the JPEGs

Gap	Delivered change	Evidence
Explicit multidimensional nutrition data	An entry-by-nutrient double[n][4] matrix now powers the Home nutrition summary from one file read.	service/NutritionService.java:123
Generated recipe favorites were not retained	The displayed-recipe favorite overload inserts a newly generated recipe into the persisted catalog before toggling it.	service/RecipeService.java:116
SDLC evidence was not assembled into a full manual	Requirements traceability, design, algorithms, flowcharts, test evidence, deployment and maintenance are recorded here.	Sections 2-14; docs/verification_summary.json

No Google Calendar implementation was added. The existing Gemini integration remains the external API example for requirement J13.

2. Purpose, users and requirements

2.1 Problem statement and objectives

Wellness records are often split between food notes, pantry lists, workout logs and daily plans. LifeSync combines these concerns in one desktop workspace. The program helps a local user maintain a profile, review estimated nutrition, organize ingredients and recipes, and schedule activities without overlapping peer events.

The objectives are to keep the core workflows usable offline, provide optional AI assistance without blocking the JavaFX thread, persist records reliably, make information readable in two themes, and demonstrate the Java programming fundamentals in the supplied academic checklist.

Actor / responsibility	Role in the system
Local user	Registers/signs in, edits profile, manages pantry and favorites, logs nutrition, schedules or completes activities, and uses chat.
Maintainer	Builds with Java 21, manages dependencies, reviews changes, tests, backups and release artifacts.
Filesystem	Stores per-entity JSON records and installation-wide theme/catalog state.
Gemini service (optional)	Accepts selected user context and returns text or structured recipe/plan suggestions over HTTPS.

2.2 Functional requirements

ID	Observable behavior
F01	Create a local account and authenticate by username or email; maintain an in-memory active user.
F02	Issue and consume hashed single-use recovery codes; reset a password only with a valid saved code.
F03	Edit body metrics, preferences and profile; calculate BMI, BMR, TDEE and the application calorie target.
F04	Create/edit/delete pantry ingredients, quantities and low-stock thresholds; filter by category.
F05	Browse/search catalog records; match pantry ingredients; generate local or AI recipes; retain favorites.
F06	Log a recipe as eaten; aggregate calories, protein, carbohydrates and fat for the selected account/day.
F07	Create/edit/reschedule/delete activities; show day/week/month views, statuses and nested sub-plans.
F08	Support 00:00-23:59 starts and 24:00 end-of-day boundaries; reject invalid or conflicting same-level intervals.
F09	Maintain chat across tab navigation without sharing its history with another account; import valid plans as a checked batch.
F10	Persist supported records and theme selection; render all existing views in light and dark modes.

2.3 Non-functional requirements and constraints

The implementation uses Java 21, JavaFX 21, Maven Wrapper and Gson. Controllers update UI controls on the JavaFX Application Thread; network work uses CompletableFuture/HttpClient and returns through Platform.runLater. JSON storage is intended for a single local application instance, not a multi-user server. Test storage is isolated under target/test-data or JUnit temporary directories. Readable category colors, wrapping advice text, keyboard focus states and accessible control labels support usability; no formal accessibility certification is claimed.

3. Java requirement traceability

The following rows transcribe the programming topics visible in Unknown.jpeg. Each is mapped to actual classes and methods. Paths are relative to src/main/java/com/lifesync/lifesync/. Line references identify the source snapshot used for this edition.

J01. Classes and Objects

Status: Covered. User, NutritionEntry, PantryItem, Recipe, Activity, ActivityTime and RecipeMatch represent the proposal domain. Food records are NutritionEntry; ingredients are PantryItem and recipe ingredient strings; local calendar events are Activity.

Source: model/User.java:23; model/Activity.java:15; service/RecipeService.java:346

J02. Arrays / ArrayLists

Status: Covered. ArrayList collections hold persisted records, recipes and chat messages. Arrays provide nutrient rows, food emojis and day names.

Source: service/DataStore.java:70; controller/ChatController.java:68

J03. Strings

Status: Covered. Names, dietary preferences, ISO dates, HH:mm times, statuses, ingredient keywords and AI text are processed as strings.

Source: model/Recipe.java:23; service/RecipeService.java:60

J04. Methods

Status: Covered. Reusable methods own calculations, validation, searching, sorting, authentication, CRUD and scheduling.

Source: service/UserService.java:84; service/ActivityService.java:225

J05. Conditionals

Status: Covered. If/switch branches validate inputs, classify BMI, select activity multipliers, route themes and reject conflicts.

Source: service/UserService.java:99; model/ActivityTime.java:23

J06. Loops

Status: Covered. Loops process entities, compare schedule intervals, expand cascade deletion, and aggregate the nutrition matrix with nested loops.

Source: controller/HomeController.java:290; service/ActivityService.java:153

J07. Searching

Status: Covered. Case-insensitive recipe name/ingredient/tag searches, category filtering, ID lookups and pantry matching are implemented.

Source: service/RecipeService.java:60; service/RecipeService.java:306

J08. Sorting

Status: Covered. Recipes sort by name, popularity, prep time or calories; activities sort by date/time; pantry items by category/name.

Source: service/RecipeService.java:221; service/ActivityService.java:45

J09. Multidimensional Data

Status: Completed in this audit. getDailyNutritionMatrix returns double[n][4]: one food-entry row and kcal/protein/carbs/fat columns. HomeController aggregates the matrix from one file snapshot. Chat context also uses List<String[]>.

Source: service/NutritionService.java:123; controller/HomeController.java:290

J10. Encapsulation

Status: Covered. Model fields are private with controlled accessors. Services own persistence and business operations; controllers own UI behavior.

Source: model/User.java:67; model/Recipe.java:74

J11. Exception Handling

Status: Covered. Validation blocks invalid input; storage failures propagate instead of false success; AI failures return local fallback results.

Source: service/DataStore.java:78; service/GeminiService.java:140

J12. File Handling

Status: Covered. Gson loads/saves JSON using UTF-8 and temporary-file replacement. Theme selection is persisted. Generated favorites now enter the catalog.

Source: service/DataStore.java:90; service/RecipeService.java:116

J13. API / External Integration

Status: Covered through Gemini; Calendar excluded. Java HttpClient integrates the existing Gemini generateContent service asynchronously. No Google Calendar API, OAuth, sync or calendar credentials are added.

Source: service/GeminiService.java:64; service/GeminiService.java:100

Favorite and Recommendation need not be standalone entity classes: favorites are a field on Recipe, while recommendations are service results. The local Activity model represents the calendar-event concept. This keeps the implementation smaller without losing the behaviors in the screenshot.

4. Software Development Life Cycle evidence

Unknown-2.jpeg specifies seven SDLC stages. The table records the activities and concrete artifacts available for this project. The documentation does not invent a historical sprint schedule, named team assignments, user-study results or a production deployment.

Stage	LifeSync evidence and handoff
1. Planning	Purpose, local-user scope, optional AI, Java/JavaFX/JSON stack and explicit integration exclusion are recorded in Sections 1-2. Roles are identified by responsibility; individual assignments are not supplied.
2. Requirements analysis	The two supplied images, J01-J13 traceability, F01-F10 functional requirements and constraints define acceptance. Google Calendar is removed from the original proposal scope by the user instruction.
3. System design	Layered architecture, data model, storage contract, UI navigation and algorithms are recorded in Sections 5-9.
4. Development / implementation	Source files implement authentication, pantry/recipe management, nutrition, activity scheduling, themes and AI. This audit added the nutrition matrix and generated-recipe favorite persistence.
5. Testing	75 current JUnit tests report zero failures/errors. Desktop checks cover themes, nutrition output, modal binding and favorite persistence. Prior regression cases are summarized separately from unit tests in Section 11.
6. Deployment	Maven Wrapper and run.sh provide repeatable local execution. Java 21 setup, environment variables and packaging steps are in Section 12. A signed installer and hosted production deployment are not delivered.
7. Maintenance / evaluation	Backups, troubleshooting, regression gates, source traceability and limitations are documented in Sections 13-14. Future work requires a defined requirement and renewed tests.

4.1 Completion and change-control rules

A change is complete when the affected behavior works through its real entry point, its data survives the required reload, the existing Maven suite passes, and the UI remains readable in both themes at the supported window sizes. A requirement change must update the traceability record, algorithms and tests. New external-calendar scope is not inferred from the original proposal.

Risk	Existing control / boundary
Unavailable or invalid AI credentials	Local recipe/advice fallbacks; requests have connection and response timeouts.
Interrupted JSON save	Temporary write followed by atomic replacement where the filesystem supports it; backups remain necessary.
Schedule conflicts and partial imports	Half-open interval checks and all-or-nothing generated-plan persistence.
UI clipping / theme regressions	Wrapping/minimum preferred heights, readable semantic colors, desktop checks and the existing FXML validation test.
Incorrect assumption about proposal implementation	Source references and an explicit supported/limited/excluded distinction in this manual.

5. System architecture and design

5.1 Layer responsibilities

LifeSync uses a JavaFX controller/service/model structure rather than a web architecture. FXML declares the existing UI hierarchy; CSS supplies appearance. SceneManager swaps authentication scenes and loads tab content into the main shell. Controllers handle events and display validation. Services own calculations, entity operations and external requests. Gson persists the model classes through DataStore.

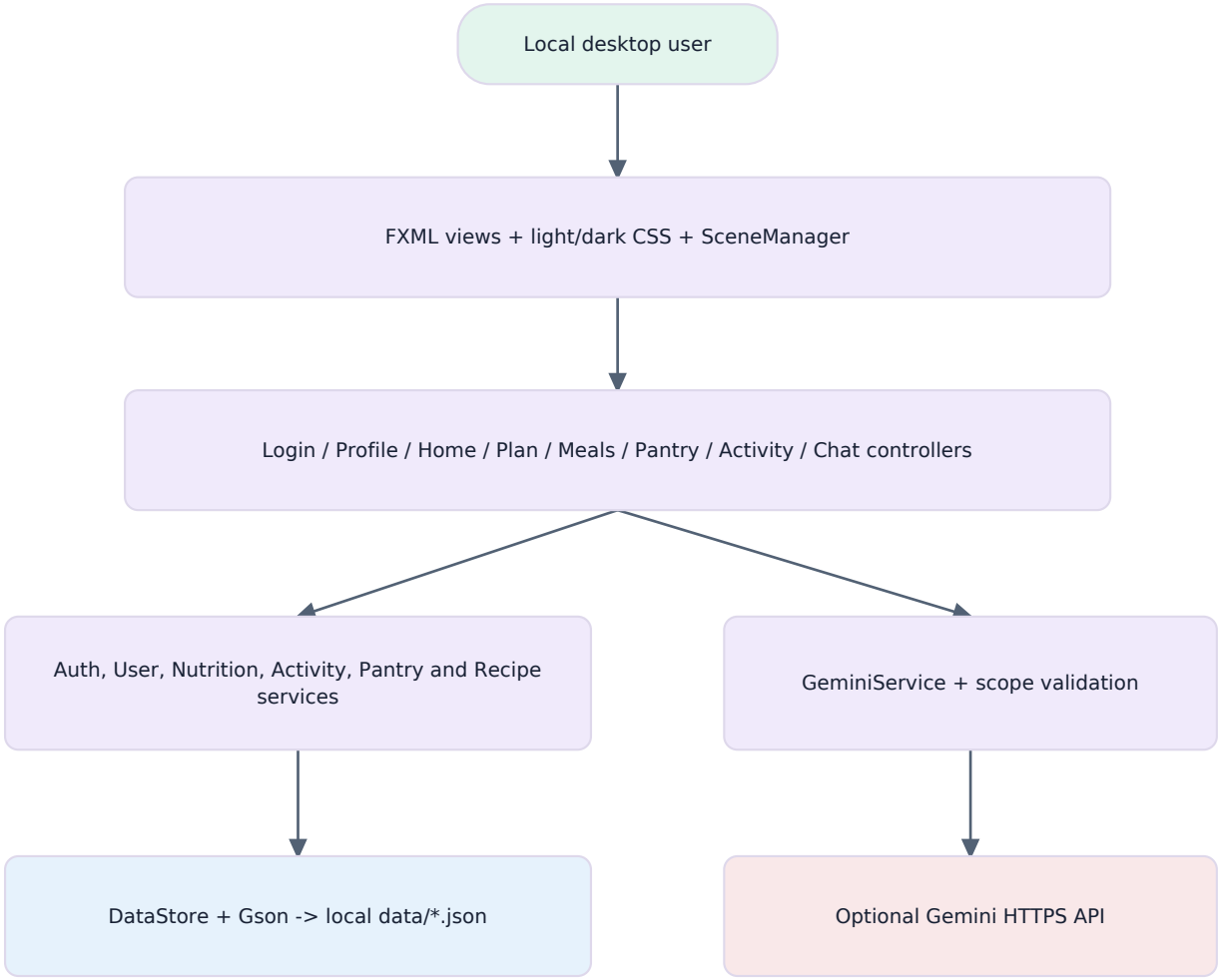


Figure 1. Component architecture. Calendar views are local; no Google Calendar component exists.

5.2 Navigation and module boundaries

Launcher starts LifeSyncApplication, which initializes SceneManager and shows Login. Successful authentication opens MainController. The main shell provides Home, Plan, Meals, Pantry, Activity and Chatbot navigation; the profile area opens My Profile. module-info.java requires javafx.controls, javafx.fxml, Gson, Ikonli, java.net.http and java.xml, and opens controller/model packages for FXML/Gson reflection.

5.3 Concurrency

File-backed UI actions run synchronously in the current desktop design. HTTP requests run asynchronously. Their completion callbacks schedule UI work through Platform.runLater. Chat tracks account ownership, a conversation generation and pending-response state so tab switches can restore the conversation and stale replies do not cross account/clear-chat boundaries. There is no cross-process file lock or general database transaction engine.

6. Data model and storage dictionary

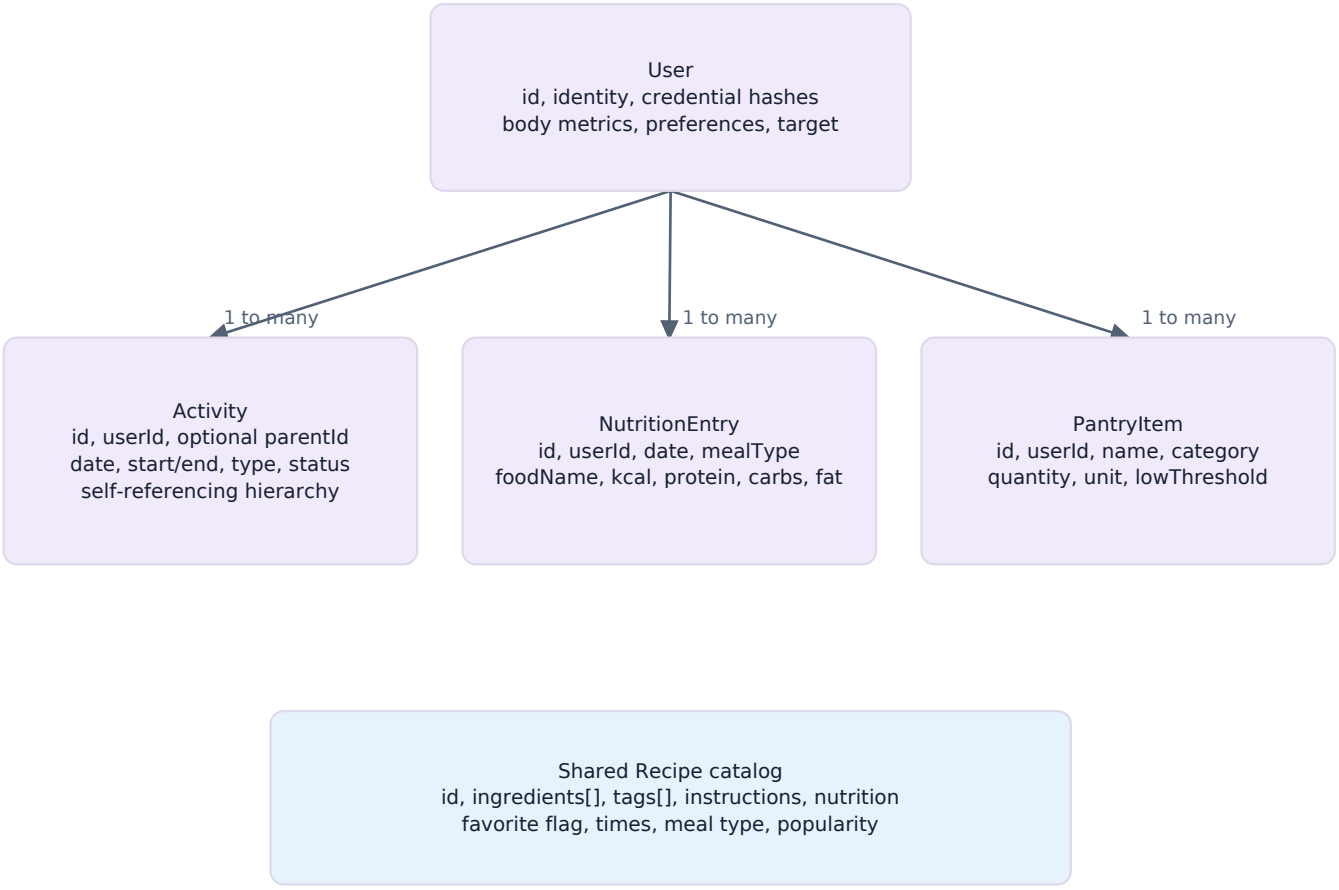


Figure 2. Main data relationships. Recipe has no userId: catalog favorite flags are currently shared by this installation.

Identifiers are UUID strings. Dates are ISO yyyy-MM-dd strings. Activity times remain HH:mm strings in JSON, including the explicit end boundary 24:00. ActivityTime is a transient validated value object; the new nutrition matrix is also transient and does not alter the stored JSON schema.

File	Contents / ownership
data/users.json	List<User>; identity, SHA-256 password hash, optional recovery-code hash and profile metrics.
data/activities.json	List<Activity>; each record has userId and may reference a parent activity.
data/pantry.json	List<PantryItem>; each record belongs to one userId.
data/nutrition.json	List<NutritionEntry>; each record belongs to one userId and one date.
data/recipes.json	Shared List<Recipe>; generated favorites are inserted here and favorite flags persist.
data/theme.json	Installation-level darkMode preference, handled by ThemeManager.
In-memory only	Active authentication session, chat history, AI response text, nutrition matrices and computed presentation state.

6.1 User fields

Field	Java type	Meaning
id	String	Unique identifier (UUID)
username	String	Login username (unique)
email	String	Email address (unique)
passwordHash	String	SHA-256 hashed password
recoveryCodeHash	String	Single-use recovery secret; null for legacy accounts
firstName	String	Display first name
lastName	String	Display last name
gender	Gender	Biological sex for BMR calc
age	int	Age in years
heightCm	double	Height in centimeters (e.g. 175.0)
weightKg	double	Weight in kilograms (e.g. 70.0)
activityLevel	ActivityLevel	Physical activity tier
dietaryPreferences	String	e.g. "Vegetarian", "High Protein", etc.
dailyCalorieTarget	int	Computed or customized kcal goal
createdDate	String	ISO-8601 date string of account creation

6.2 Activity fields

Field	Java type	Meaning
id	String	Unique identifier (UUID)
userId	String	Owner of this activity (references User.id)
title	String	Short display name, e.g. "Morning Workout"
description	String	Optional longer description
type	ActivityType	Category of the activity
date	String	ISO date string (yyyy-MM-dd)
startTime	String	Start time string (HH:mm), e.g. "07:00"
endTime	String	End time string (HH:mm), including "24:00" for end of day
status	Status	Current lifecycle status
caloriesBurned	int	Estimated calories burned (for workouts)
parentId	String	Optional parent activity ID for nested sub-schedules

6.3 PantryItem fields

Field	Java type	Meaning
id	String	Unique identifier (UUID)
userId	String	Owner of this item (references User.id)
name	String	Ingredient name, e.g. "Chicken Breast"
category	String	Grouping, e.g. "Meat", "Vegetables", "Spices"
unit	String	Unit of measure, e.g. "g", "kg", "pieces", "ml"
quantity	double	Current amount on hand
lowThreshold	double	Below this quantity, item is flagged as "low"
expiryDate	String	Optional expiry date (yyyy-MM-dd), may be null

6.4 NutritionEntry fields

Field	Java type	Meaning
id	String	Unique identifier (UUID)
userId	String	Owner of this entry (references User.id)
date	String	ISO date string (yyyy-MM-dd)
mealType	MealType	Meal category
foodName	String	Description of what was eaten
calories	double	Total kilocalories
proteinG	double	Grams of protein
carbsG	double	Grams of carbohydrates
fatG	double	Grams of fat

6.5 Recipe fields

Field	Java type	Meaning
id	String	Unique identifier (UUID)
name	String	Recipe name, e.g. "Teriyaki Chicken Bowl"
description	String	Brief description of the dish
ingredients	List<String>	List of required ingredients with amounts
instructions	String	Step-by-step cooking instructions
cookingMethod	String	e.g. "Stir-fry", "Bake", "Grill"
prepTimeMinutes	int	Preparation time in minutes
cookTimeMinutes	int	Cooking time in minutes
calories	double	kcal per serving
proteinG	double	grams of protein per serving
carbsG	double	grams of carbs per serving
fatG	double	grams of fat per serving
tags	List<String>	Categorical tags, e.g. "High Protein", "Quick"
favorite	boolean	Whether the user has marked this as a favorite
mealType	String	e.g. "Breakfast", "Lunch", "Dinner", "Snacks"
imagePath	String	Resource path to recipe image, e.g. "/com/lifesync/lifesync/images/grilled_chicken_bowl.jpg"
popularity	int	Popularity rating for sorting (e.g. 1-100)

6.6 Representative synthetic JSON

The following record is illustrative. It is not copied from real account data. Omitting parentId represents a legacy/top-level activity and deserializes as null.

```
{
  "id": "example-child-id", "userId": "example-user-id",
  "parentId": "example-parent-id", "title": "Evening reading",
  "type": "STUDY", "date": "2026-10-24",
  "startTime": "23:00", "endTime": "24:00",
  "status": "UPCOMING", "caloriesBurned": 0
}
```

7. Algorithms and pseudocode

7.1 Account registration, login and recovery

Registration validates visible form fields, checks duplicate username/email case-insensitively, hashes the password and persists a User. Login compares the submitted password hash with the stored hash and sets the in-memory current user only on success. Recovery requires a previously saved high-entropy code; knowing an email address alone does not authorize a reset.

```
REGISTER(form):
  validate required identity fields and finite body metrics
  LOAD users
  IF username OR email already exists (case-insensitive): reject
  user.passwordHash = SHA256(form.password)
  calculate daily target; SAVE user
  issue recovery code after verifying the new credentials
  show the code once; return to Login

LOGIN(identifier, password):
  LOAD users
  FIND a user matching username OR email
  IF found AND SHA256(password) equals stored passwordHash:
    currentUser = user; open dashboard
  ELSE show authentication error

RESET(identifier, code, newPassword):
  validate newPassword length and non-blank value
  FIND user matching identifier
  IF recoveryCodeHash is absent OR does not match SHA256(trim(code)): reject
  replace passwordHash; remove recoveryCodeHash
  SAVE all users; clear that user's active session
  tell user to sign in and generate a replacement recovery code
```

A new recovery code contains 32 SecureRandom bytes encoded with URL-safe Base64. Only its SHA-256 hash is saved. Issuing a replacement code invalidates the previous one; successful reset consumes the code.

7.2 BMI, BMR, TDEE and target computation

Quantity	Implementation rule
BMI	weightKg / (heightCm / 100)^2; rounded to one decimal. Non-positive/missing inputs return 0.
BMI categories	Below 18.5: underweight; 18.5 to below 25: normal; 25 to below 30: overweight; 30 and above: obese. The UI shows Pending when metrics are absent.
BMR	10*weightKg + 6.25*heightCm - 5*age + sex adjustment. Adjustment is +5 for MALE and -161 for FEMALE.
TDEE	BMR multiplied by the stored activity-level multiplier.
Multipliers	Sedentary 1.2; lightly active 1.375; moderately active 1.55; very active 1.725; extra active 1.9; null defaults to 1.2.
Calorie target	Round TDEE. Underweight: add 250. Overweight/obese: subtract 350, with floor 1400. Normal: use rounded TDEE.
Displayed weight range	18.5*(heightM^2) through 24.9*(heightM^2), formatted to one decimal.

```
CALCULATE_PROFILE(user):
  IF height, weight or age needed by a formula is missing: show pending values
  bmi = round(weightKg / square(heightCm / 100), 1 decimal)
  category = classify(bmi)
  bmr = 10*weightKg + 6.25*heightCm - 5*age + sexAdjustment
  tdee = bmr * activityMultiplier
  target = rounded tdee, adjusted by the program's category rule
  persist user; refresh profile, advice and dashboard values
```

These are the rules implemented in UserService, not a personalized clinical assessment. Nutrition and AI outputs are estimates. Age-specific clinical interpretation, pregnancy adjustments and medical diagnosis are outside this desktop implementation.

7.3 Multidimensional nutrition aggregation

Let n be the number of this account's logged food entries on the requested date. The matrix has n rows and four numeric columns. Columns have different units, so each is summed separately. Empty input returns zero rows and the dashboard starts from four zero totals.

Row example	Calories (kcal)	Protein (g)	Carbs (g)	Fat (g)
Food entry 1	420	35	40	12
Food entry 2	180	10	20	8
Column totals	600	45	60	20

```
DAILY_NUTRITION_MATRIX(userId, date):
  entries = LOAD and FILTER nutrition records by userId and date
  matrix = new double[entries.count][4]
  FOR row FROM 0 TO entries.count - 1:
    matrix[row] = [calories, proteinG, carbsG, fatG] from entries[row]
  RETURN matrix

HOME_NUTRITION_OVERVIEW(userId, date):
  matrix = DAILY_NUTRITION_MATRIX(userId, date)
  totals = [0, 0, 0, 0]
  FOR EACH row IN matrix:
    FOR column FROM 0 TO 3:
      totals[column] += row[column]
  macroKcal = 4*proteinG + 4*carbsG + 9*fatG
  IF macroKcal > 0:
    percent = 100 * each macro's kcal / macroKcal
  ELSE all macro percentages = 0
  update calorie count, grams, percentages and colored bars
```

The macro bars show each macro's share of macro-derived calories, not progress toward a protein/carbohydrate/fat goal. Total food calories may differ from 4/4/9-derived macro calories. The matrix replaces four repeated nutrition-file reads in the Home summary with one snapshot.

7.4 Time ranges and conflict detection

ActivityTime converts start/end text into minutes after midnight. A start is 00:00-23:59; an end may also be 24:00 (1440 minutes). The interval is half-open: [start, end). Adjacent activities are valid. A plan ending at 24:00 stays on its stored date and does not overlap the next date's 00:00 activity.

```
PARSE_INTERVAL(startText, endText):
  require strict HH:mm start in 00:00..23:59
  start = 60*hour + minute
  end = 1440 IF endText equals "24:00" ELSE parse strict 00:00..23:59
  require start < end
  RETURN [start, end)

CONFLICTS(candidate, existing):
  IF different user, date or scheduling level: false
  IF existing is cancelled OR is the excluded edit ID: false
  RETURN candidate.start < existing.end AND existing.start < candidate.end

SAVE_EVENT(candidate):
  validate interval; for a sub-plan, validate parent bounds
  compare against peer activities using CONFLICTS
  IF any conflict: reject and explain
  ELSE persist and redraw the selected date
```

Example	Expected result
22:00-23:00 beside 23:00-24:00	No conflict: touching boundary only.
23:00-24:00 beside 23:30-23:59	Conflict: overlapping intervals.
23:00-00:00 on the same date	Rejected: use 24:00 for the midnight end boundary.
24:00 as a start, or 24:01 as an end	Rejected by strict interval parsing.
Sub-plan inside its parent	Allowed against the parent; overlapping siblings are checked.

7.5 Atomic generated plans and cascade deletion

```
ADD_GENERATED_PLAN(proposed):
  all = LOAD activities
  merged = copy(all); usedIds = IDs from all
  FOR EACH candidate IN proposed:
    validate owner, title, ISO date and interval
    reject duplicate ID
    compare against merged same-user, same-date, same-parent peers
    IF conflict: reject ENTIRE plan before any save
    append candidate to merged
  SAVE merged once

DELETE_WITH_DESCENDANTS(activityId):
  all = LOAD activities; removedIds = {activityId}
  REPEAT:
    changed = false
    FOR EACH activity IN all:
      IF activity.parentId IN removedIds:
        changed = add activity.id to removedIds OR changed
  UNTIL no IDs were added
  REMOVE all records whose ID is in removedIds
  SAVE once
```

The deletion set handles arbitrary descendant depth and terminates even if malformed data contains a parent cycle. Legacy records without parentId remain top-level. Deletion uses the selected button action rather than a special user-entered title such as DELETE.

7.6 Searching, sorting, recipe matching and favorites

```
SEARCH_RECIPES(query):
  normalize query to lowercase and trim whitespace
  LOAD catalog; retain recipes whose name, ingredients or tags contain query
  apply requested service filters and comparator when used

MATCH_PANTRY(userId):
  pantryNames = lowercase names in the user's pantry
  FOR EACH catalog recipe WITH at least one ingredient:
    available = count ingredient strings matched by substring in either direction
    missing = ingredient strings with no pantry-name match
    score = 100 * available / ingredientCount
    collect [recipe, score, missing]
  sort matches by descending score

TOGGLE_DISPLAYED_FAVORITE(recipe):
  LOAD/initialize catalog
  FIND catalog record with recipe.id
  IF missing: append the displayed generated recipe
  toggle favorite; SAVE catalog
  IF save fails: restore old state and report the error
  RETURN persisted favorite state
```

Matching is name/substring based; it is not a quantity-aware nutrition planner or allergen certification system. RecipeService exposes filters/sort methods beyond those surfaced by every current screen. The Add Recipe dialog searches its permitted catalog, which favors saved favorites and pantry-match records. The primary Meals cards are generated from current in-stock pantry items. Favorites are catalog-level flags shared by the installation; separate per-account favorites are not implemented.

7.7 AI request and chat response lifecycle

```
REQUEST_AI(userContext, prompt):
  prepare a local fallback first
  build JSON request for Gemini generateContent
  dispatch asynchronously with connection and request timeouts
  IF HTTP success AND response parses into usable content: return content
  ELSE return local fallback

SEND_CHAT(message):
  reject empty input or a second send while awaiting a reply
  record user turn; refuse unsupported topics locally
  capture account ID and conversation generation
  request AI asynchronously
  ON JavaFX thread when completed:
    ignore obsolete generation or mismatched active account
    record reply and valid parsed plan tokens
    render in the current chat controller and enable input

CLEAR_CHAT_OR_SWITCH_ACCOUNT:
  clear applicable history; increment generation; clear pending state
  late responses from the older generation cannot repopulate the new conversation
```

The configured model string is read from GeminiService in the source. Access depends on the configured key and provider availability; a passing fallback-tolerant unit test does not prove the remote account has quota. Chat history lasts for the running process and tab navigation, not across application restarts.

7.8 JSON write algorithm and complexity

```
SAVE_ALL(filename, items):
  create data directory if necessary
  create a temporary file in that directory
  serialize items as pretty-printed UTF-8 JSON
  close the writer
  atomically replace destination when supported
  otherwise replace destination using the filesystem fallback
  on I/O failure: throw an error to the caller
  always attempt temporary-file cleanup
```

Operation	Approximate cost (excluding string length and disk latency)
Daily nutrition matrix	O(N) filtering of N stored entries plus O(4n) matrix fill/aggregation for n selected entries.
Recipe name sorting / search	O(R log R) catalog sort, plus a scan of recipe fields and ingredients for searching.
Pantry matching	O(R*I*P + R log R): R recipes, average I ingredients, P pantry names.
Conflict lookup	Loads activities; user activities are sorted by date/time, then the selected day is scanned.
Generated-plan batch	O(B*A + B^2) comparisons for B proposed and A existing activities, followed by one save.
Cascade delete	O(A*H) fixed-point scans in the worst case for A activities and hierarchy depth H.

8. Flowcharts and process diagrams

Notation: rounded green boxes are start/end states; purple boxes are processing steps; amber diamonds are decisions; blue boxes are stored data. Arrows indicate control or data movement. These are vector diagrams embedded in the PDF, so they remain sharp when printed.

8.1 Authentication flowchart

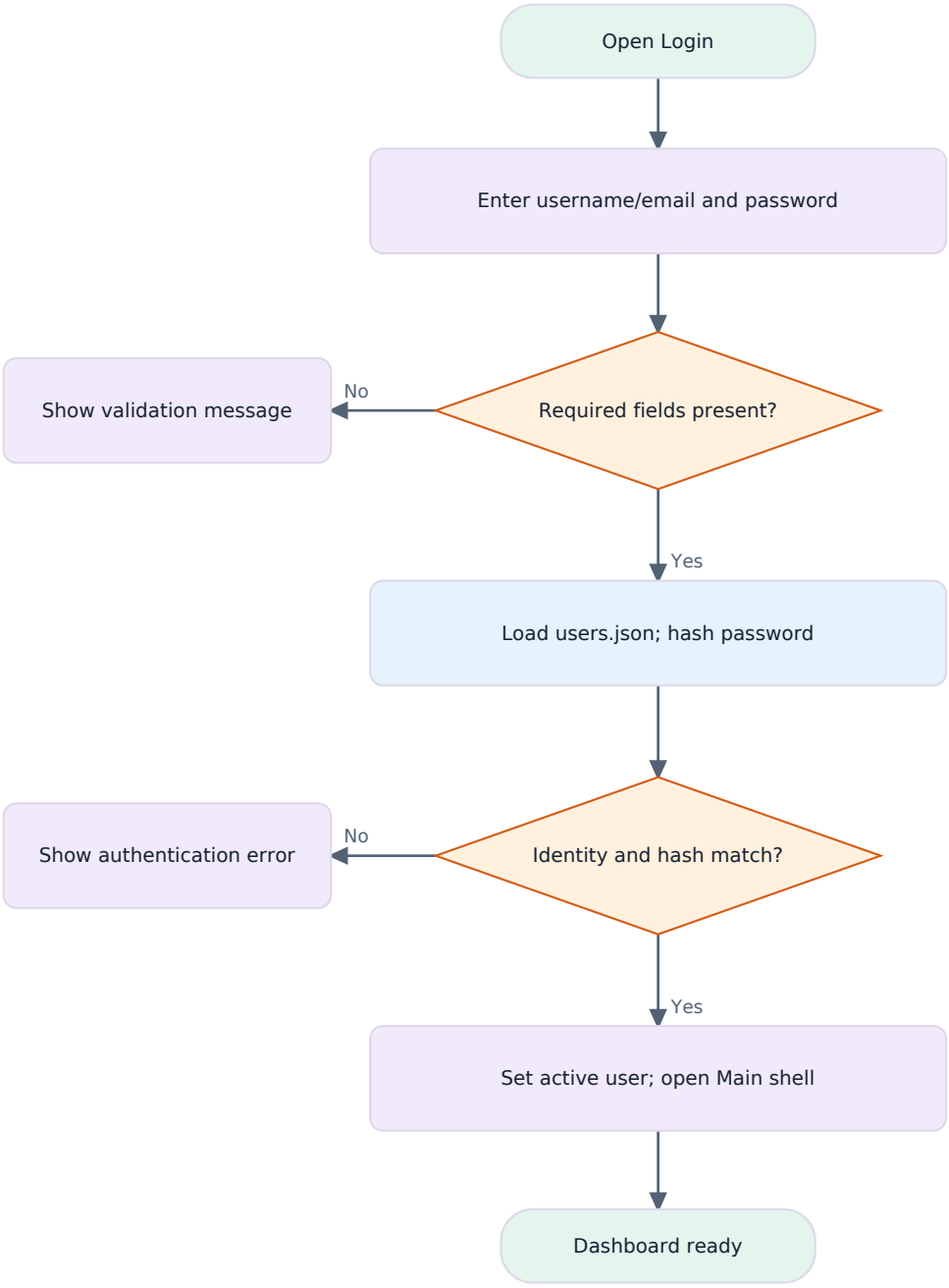


Figure 3. Login validation and session creation. Rejected inputs remain outside the authenticated shell.

8.2 Activity scheduling flowchart

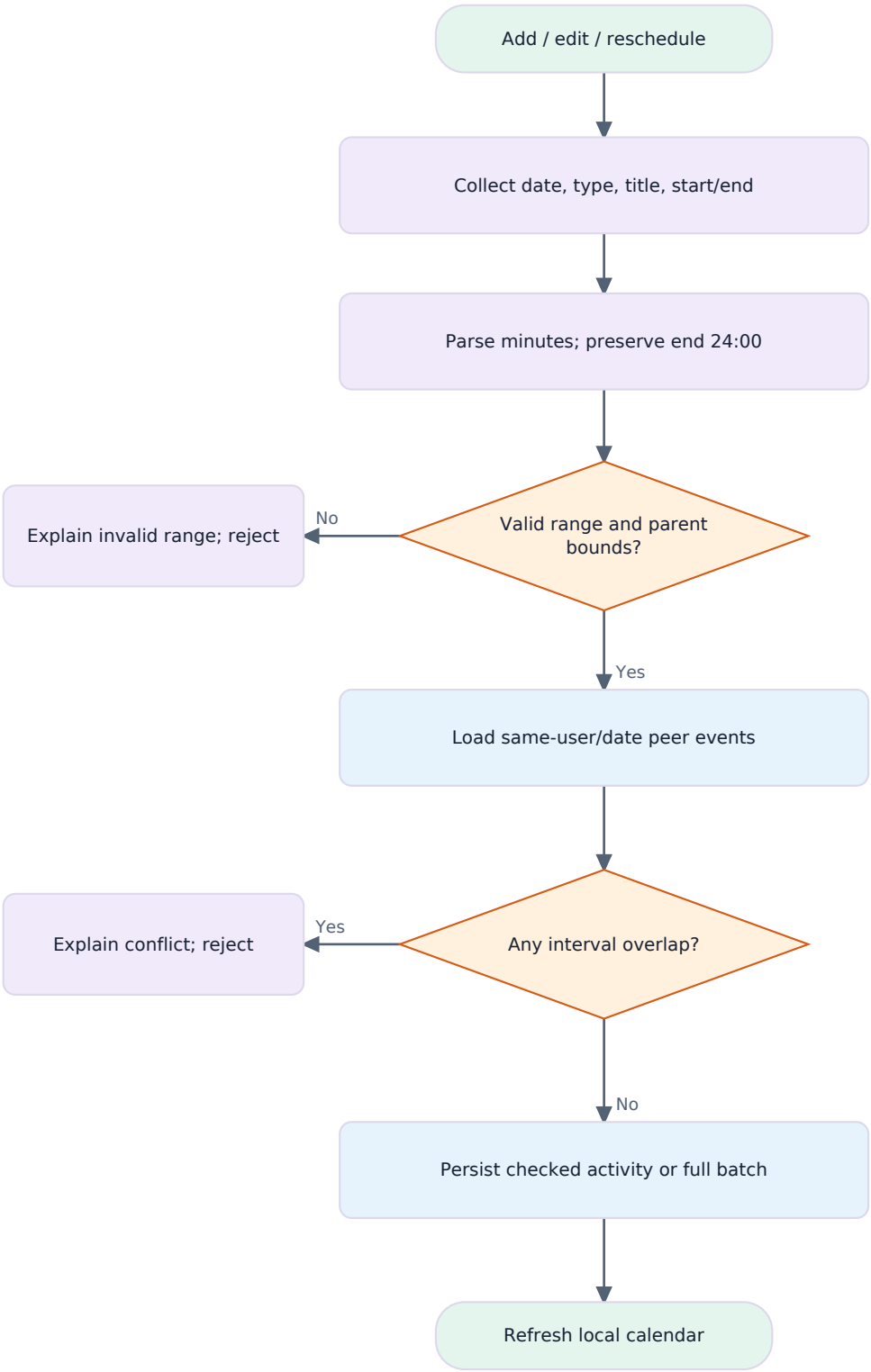


Figure 4. Validated local scheduling. Google Calendar is not a branch of this flow.

8.3 Nutrition matrix flowchart

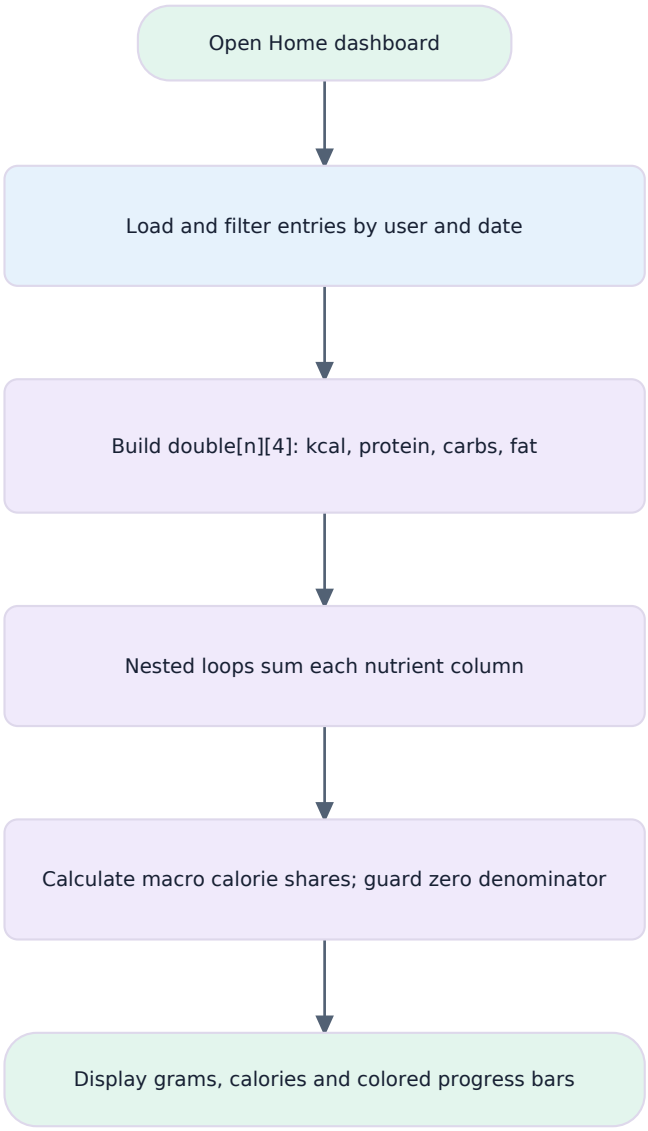


Figure 5. The multidimensional data requirement is connected to real dashboard output.

8.4 AI-assisted recommendation flowchart

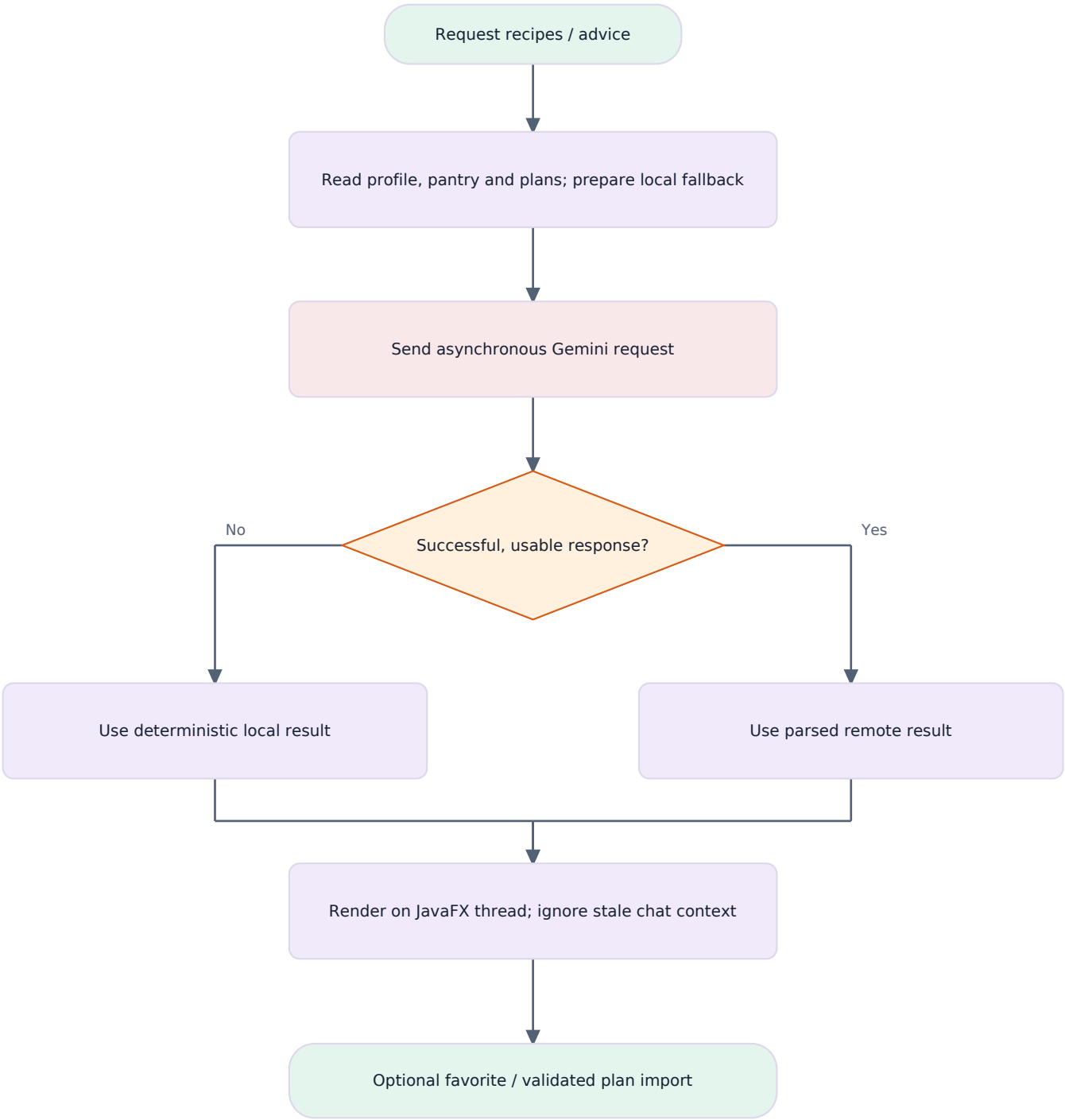


Figure 6. AI is optional. Local recommendation and scheduling workflows remain available when the network fails.

9. User interface and operating guide

9.1 Home and visual language

Home combines summary cards, today's activities, nutrition, advice and upcoming plans. Protein uses violet, carbohydrates amber, fat mint/emerald and calories coral. Selected navigation uses a theme-specific gradient. BMI category badges use paired foreground/background colors rather than inheriting generic dark-mode label colors.

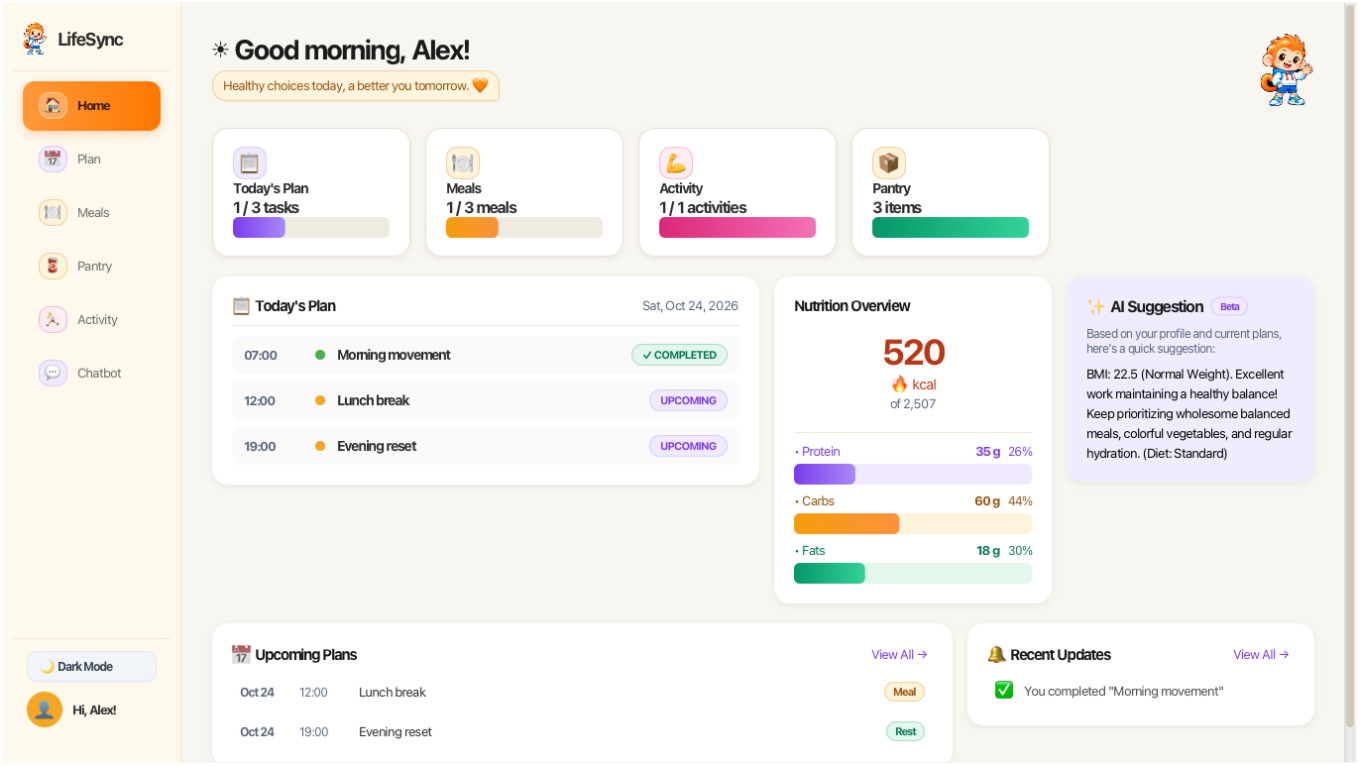


Figure 7. Home in light mode, using a synthetic demonstration account.

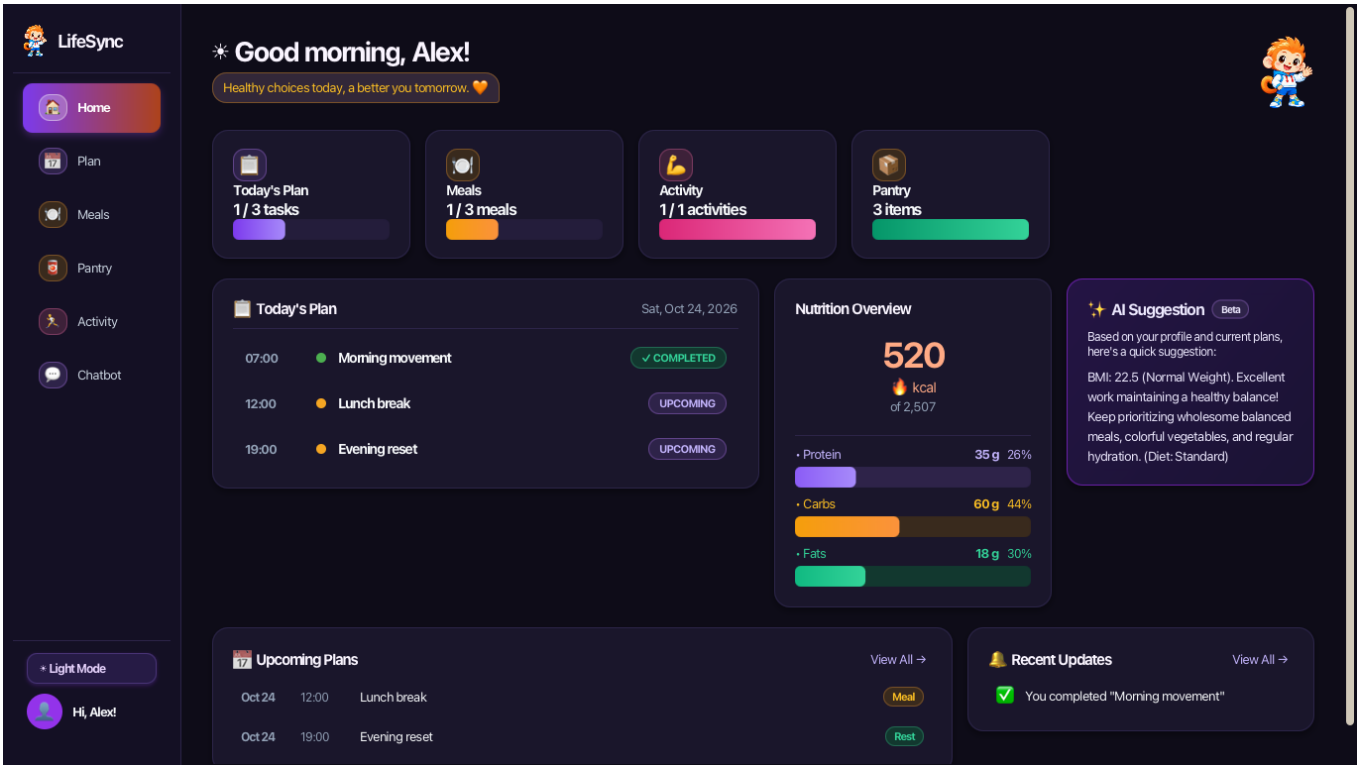


Figure 8. The same navigation and nutrition hierarchy in dark mode.

9.2 Account and profile workflow

Select Sign up, enter the identity/password fields and body metrics, then create the account. Copy the generated recovery code into a private safe place. Sign in using username or email. Open My Profile through the profile area in the sidebar to change metrics or preferences. Save Profile recalculates the target. Account recovery code verifies the current password before replacing a saved code.

Forgot password asks for username/email, a previously saved recovery code, the new password and confirmation. An existing account without a recovery code must enroll while it can still sign in. The program does not send recovery email. The Remember me checkbox is present in the login view, but persistent automatic sign-in is not implemented.



Figure 9. Correctly paired dark-mode BMI category colors. The numerical value and category stay distinct.

9.3 Pantry and meals workflow

Open Pantry and add an ingredient, quantity, unit, category and low-stock threshold. Quantities and thresholds must be finite and non-negative. Invalid input keeps the ingredient dialog open. Editable category and unit text are retained. Use edit/delete actions to maintain stock.

Open Meals to see locally generated pantry recipes immediately, with optional Gemini enhancement. A recipe card opens its detail modal. Favorite saves the displayed recipe into the catalog if it was newly generated. The Add Recipe catalog searches permitted favorites/pantry matches. Add to meal plan collects a meal category, date and time range and blocks conflicts; Log as Eaten Today creates a NutritionEntry.

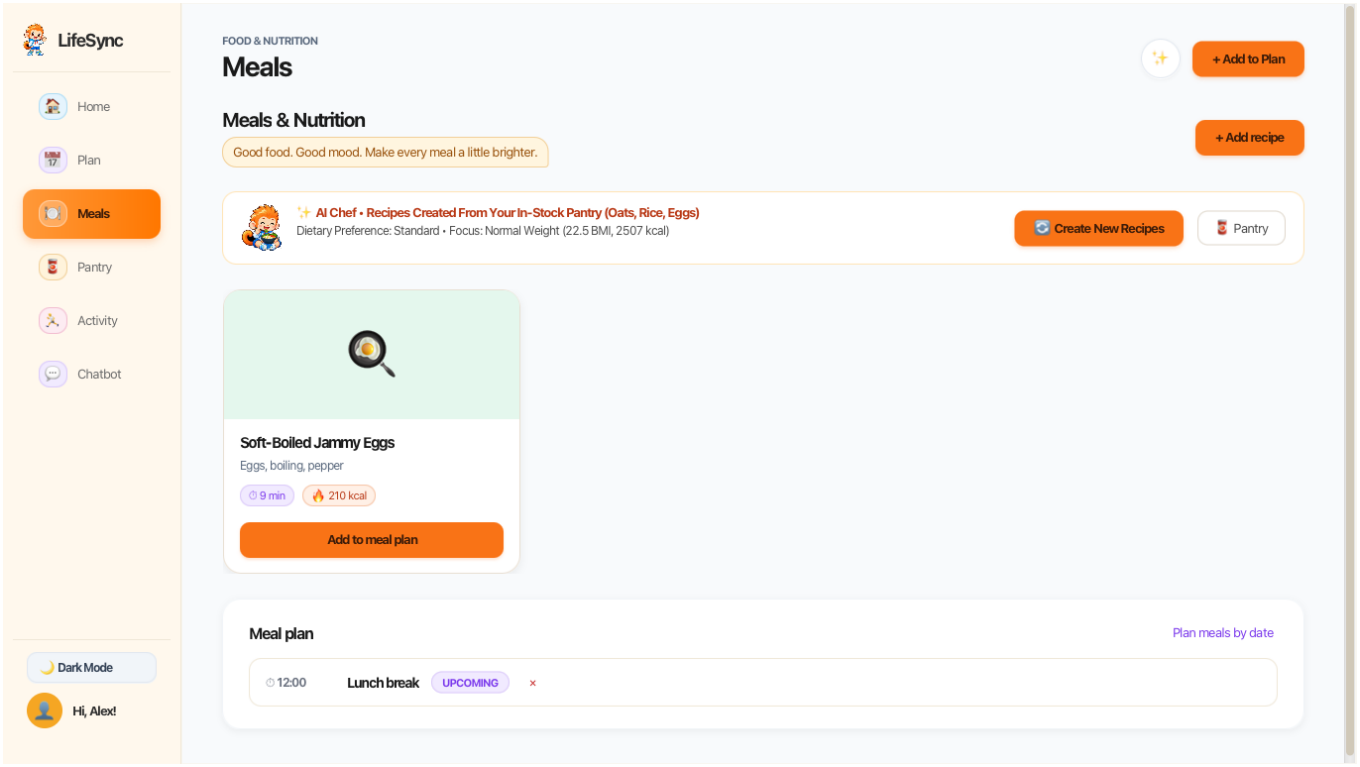


Figure 10. Pantry-driven recipe cards and meal planning in light mode.

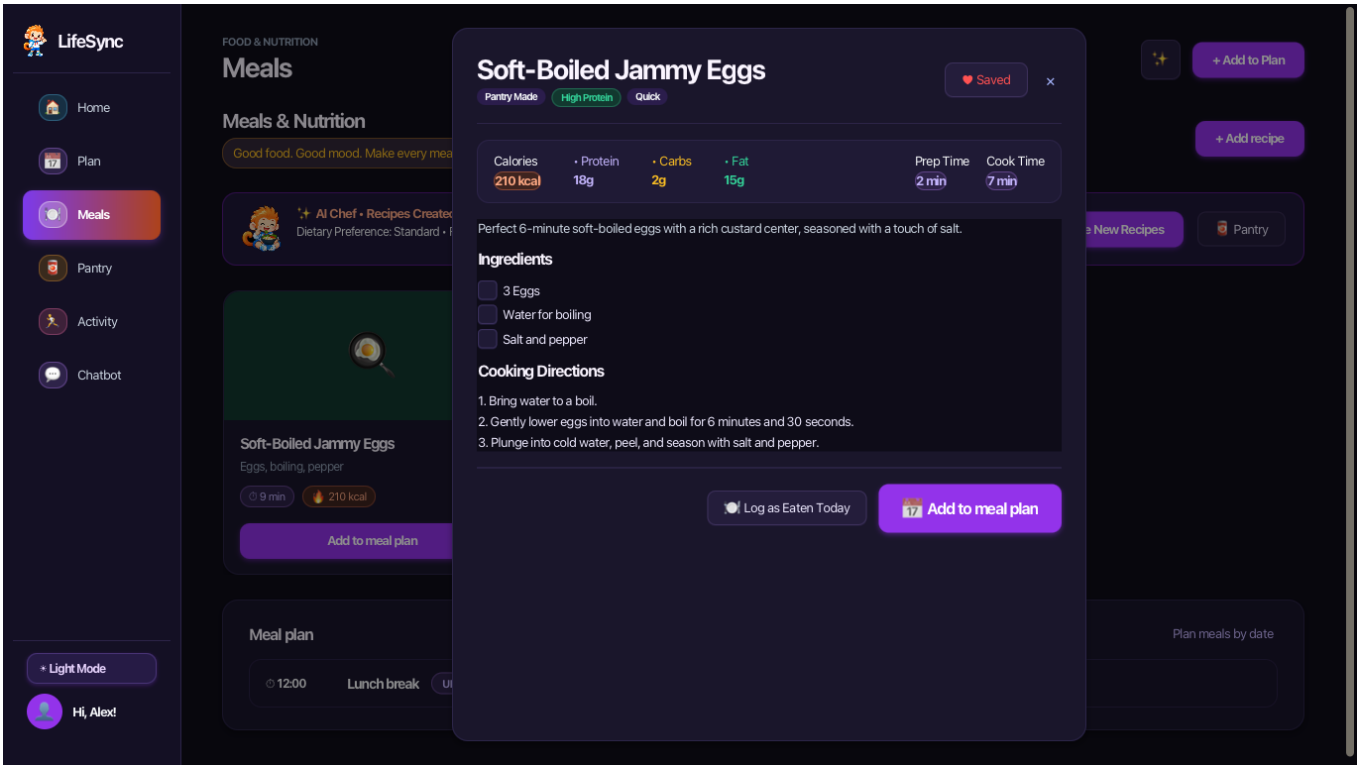


Figure 11. Recipe details, macro colors and a persisted favorite in dark mode.

9.4 Plan and Activity workflow

Open Plan for the local Day, Week or Month view. The Day view starts in full-day mode, from 12 AM through 11 PM. Use 24:00 as an end time, for example 23:00-24:00. Add a sub-plan from its parent block and keep its interval inside the parent. Use Reschedule, edit, complete, cancel or delete controls as appropriate. Deleting a parent removes its descendants.

Week event titles wrap. Month cells use compact previews and counts; hover a preview for the full title/time or open the day to inspect all activities. Narrow windows can scroll the local calendar horizontally rather than squeeze critical controls. Activity shows workout summaries and a filterable list; its BMI advice grows to its required height

and scrolls with the page instead of being cut to one line.

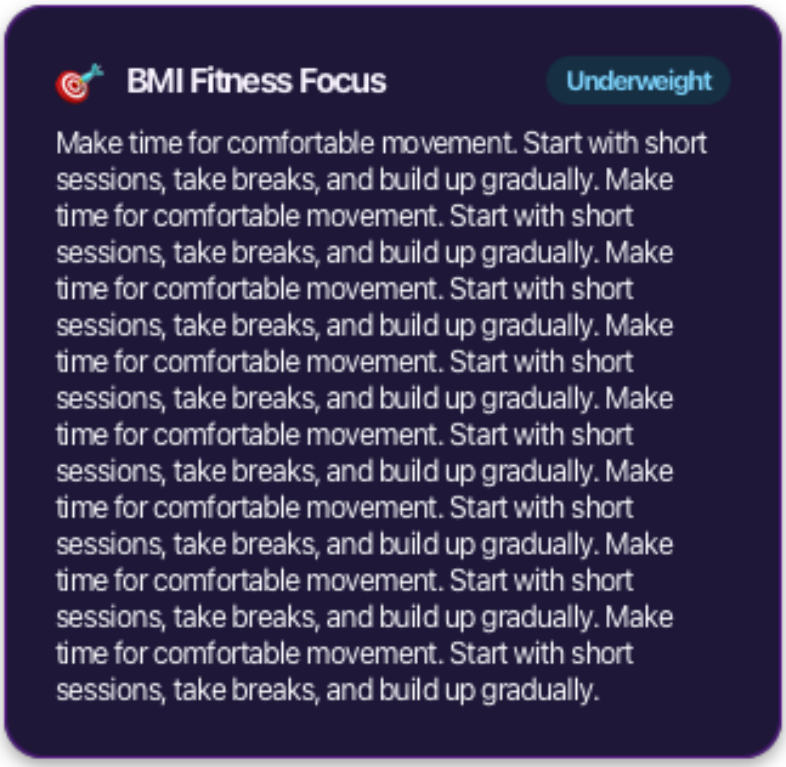


Figure 12. A deliberately long synthetic advice message remains fully readable.

9.5 Chat and themes

Chatbot accepts LifeSync-related planning, pantry, nutrition and fitness questions. Unsupported topics receive a local refusal. Recognized event tokens produce an actionable plan card; Add All validates the entire set before any write. A conflict leaves all items unsaved. Clear Chat invalidates pending replies. Switching accounts clears the previous account's displayed conversation. The sidebar theme toggle changes the current scene; theme selection is saved for the installation.

10. Validation, exceptions and security boundaries

Condition	Expected response
Empty login fields / bad credentials	Visible login message; authentication does not advance.
Invalid signup/profile numeric input	Field-level/status feedback; NaN and infinite values do not enter the save path.
Negative or non-finite pantry stock	Dialog remains open; service validation rejects invalid additions/updates before persistence.
Invalid start/end or parent bounds	A themed time-validation error; no invalid plan write.
Overlapping peer activities	A conflict message; Activity, Plan and Meals reject the scheduling request.
AI plan with a conflict/duplicate/malformed interval	The whole batch is rejected before the JSON write; the plan remains available for correction.
AI timeout / invalid key / parse failure	A local fallback result; the UI remains responsive.
Unreadable JSON / failed save	Storage error propagation; missing files are different from unreadable files. Corrupt data must not be treated as an empty account store.
Invalid / consumed recovery code	Reset is rejected without changing the password or consuming another valid code.

10.1 Credential and data handling

Passwords use unsalted SHA-256 in the existing educational implementation. This is not a modern password-hardening scheme such as a salted password KDF. Recovery secrets use SecureRandom and are stored only as hashes; this does not encrypt the remaining JSON data. A person with filesystem access can read or alter local records. Account separation is enforced by service filters and UI/session logic, not operating-system isolation.

GEMINI_API_KEY is read from the process environment. run.sh loads the local .env file; Java itself does not automatically load .env. The supplied .gitignore excludes environment files. The implementation sends selected user context to Gemini when a request is made, so an operator should use appropriate demonstration data for presentations. No external calendar account, OAuth token or Google Calendar permission is required.

10.2 Persistence and session boundaries

DataStore writes to the directory named by lifesync.data.dir, defaulting to data relative to the working directory. Tests override this location. Atomic replacement protects one file at a time; fallback filesystems may provide only replacement semantics. There is no inter-process lock, cross-file transaction, encrypted database, or cloud backup. Run one application instance against one data directory.

11. Verification and test evidence

The current Maven reports contain **75 tests, zero failures and zero errors**. Counts below are read from target/surefire-reports rather than estimated. Re-run the suite after any source change and regenerate this document for a new snapshot.

JUnit suite	Tests	Failures	Errors	Skipped
ActivityTimeTest	5	0	0	0
FxmlValidationTest	1	0	0	0
MascotAssetTest	1	0	0	0
ActivityServiceTest	21	0	0	0
AuthServiceTest	9	0	0	0
ChatScopeValidatorTest	4	0	0	0
DataStoreTest	3	0	0	0
GeminiServiceTest	7	0	0	0
PantryServiceTest	5	0	0	0
RecipeServiceTest	8	0	0	0
RequirementCoverageTest	7	0	0	0
ThemeManagerTest	4	0	0	0

11.1 Requirement-audit regression cases

Case	Expected / verified behavior
Matrix shape and column mapping	Multiple daily entries retain kcal/protein/carbs/fat in the documented columns.
Matrix account/date isolation	Another account or date does not contribute to the requested snapshot.
Matrix consistency	Column sums agree with the existing four total getters.
Matrix ownership	Changing the returned array cannot modify saved nutrition records.
Generated favorite reload	A new generated recipe is present and favorited after a catalog reload.
Generated unfavorite / duplicate prevention	Unfavoriting persists without inserting a duplicate ID.
Existing catalog favorite	Toggling a catalog recipe retains the rest of the catalog.

11.2 Desktop verification and limits of the evidence

Desktop checks exercised the existing navigation in both themes, macro totals and gradients, the recipe detail modal, live recipe-backdrop theme changes and generated favorite persistence. Earlier targeted regression checks covered all four BMI classes plus Pending, long advice text, two window sizes, pantry form recovery, the DELETE-title regression, late-night scheduling and chat account isolation. A 42-case screen sweep checked labels/buttons and Calendar Day/Week/Month layouts. Compact month previews intentionally use ellipsis with full-text tooltips.

The Maven FXML test proves well-formed XML, not every rendered layout. Gemini tests accept useful fallback output, so their success does not certify remote quotas, medical correctness or cloud availability. Desktop checks were performed on this macOS workspace; Windows packaging and every display/font combination were not independently certified.

11.3 Suggested acceptance demonstration

Step	Pass condition
Register a synthetic account; save recovery code	Account can sign in and the code is displayed once.
Set 170 cm and 48 kg; switch to dark mode	BMI category is readable and advice remains fully accessible.
Add pantry stock and inspect Meals	Local recipes appear; favorite one, reload catalog, and find it saved.
Log two foods	Home totals equal the matrix column sums; macro shares sum approximately to 100% after rounding.
Schedule 23:00-24:00, then attempt 23:30-23:59	First saves; overlapping peer is rejected; next day 00:00 is independent.
Create parent, child and grandchild; delete parent	All descendants disappear while unrelated events remain.
Import an AI plan containing one conflict	No proposed items are partially inserted.
Switch accounts after a chat exchange	The prior account's message is not shown in the new account.
Restart application	Persistent records/theme remain; active login and in-memory chat are recreated as documented.

12. Installation, execution and deployment

12.1 Runtime prerequisites

Component	Repository configuration
Java compiler/runtime	Java 21; maven-compiler-plugin uses release 21.
JavaFX	javafx-controls and javafx-fxml 21.0.6.
Persistence	Gson 2.11.0; writable local data directory.
Icons	Ikonli JavaFX / FontAwesome5 pack 12.3.1.
Tests	JUnit Jupiter 5.12.1; Maven Surefire 3.2.5.
Build wrapper	Maven Wrapper with the distribution configured in .mvn/wrapper/maven-wrapper.properties.
Network	Needed for initial Maven dependency downloads and optional Gemini requests, not local CRUD after setup.

12.2 macOS / Linux-style shell commands

```
# Run from the project directory with a Java 21 JDK selected:
sh mvnw test
sh mvnw javafx:run

# This repository's macOS launcher selects the installed Java 21 and loads .env:
./run.sh

# Generic explicit selection when Java 21 is not the default:
export JAVA_HOME=/path/to/jdk-21
export PATH="$JAVA_HOME/bin:$PATH"
sh mvnw test
```

12.3 Windows PowerShell commands

```
cd path\to\LifeSync
$env:JAVA_HOME = "C:\path\to\jdk-21"
$env:Path = "$env:JAVA_HOME\bin;$env:Path"
.\mvnw.cmd test
.\mvnw.cmd javafx:run

# Optional AI key for the current process:
$env:GEMINI_API_KEY = "YOUR_KEY_HERE"
.\mvnw.cmd javafx:run
```

Use placeholders above only as instructions; never put a real API key in submitted documentation. On Windows or in an IDE, set GEMINI_API_KEY in the run configuration/environment. The macOS run.sh helper is not a Windows .env loader. If JAVA_HOME is unset or Java 8 is active, select a Java 21 JDK before building.

12.4 Build and release handoff

```
sh mvnw clean test
sh mvnw -DskipTests package
# Primary runnable development entry point:
sh mvnw javafx:run
```

The ordinary Maven JAR is not claimed to be a standalone executable bundle with every JavaFX native library. Use the configured JavaFX launch path for demonstration. A platform-specific installer or runtime image needs its own packaging and target-OS test. Back up data before replacing an application build and preserve the working-directory convention so the same local data folder is used.

13. Operations, maintenance and troubleshooting

13.1 Backup, restore and data ownership

Close the application before taking or restoring a consistent backup. Copy the full data directory to a dated backup location. To restore, preserve the current directory as a separate backup, replace data with the known-good copy, then reopen and verify a synthetic/read-only workflow first. Do not edit live JSON while the application is saving, and do not run multiple instances against the same records.

13.2 Troubleshooting matrix

Symptom	Check / corrective action
Unsupported release / compile failure	Confirm java -version and JAVA_HOME select Java 21, then re-run the Maven wrapper.
AI output is local or unavailable	Check that the actual run process received GEMINI_API_KEY, network access and provider availability. Local fallback remains expected behavior.
23:00-00:00 is rejected	Use 23:00-24:00 for the end of the same stored date.
Plan cannot be added	Review same-day peers, sibling sub-plans and duplicates. A generated batch is rejected as a whole.
Forgot-password reset fails	Use the saved, unused recovery code for that account. An email address alone cannot reset it.
Records appear missing	Check the process working directory and lifesync.data.dir. Test data and production data are intentionally separate.
JSON read/write error	Check permissions, free space and file validity; preserve the faulty file and restore a known-good backup rather than replacing it with an empty list.
Category or advice appears cut off	Use the updated styles/FXML, restart the application and inspect both themes/window sizes before changing layout constraints.

13.3 Documentation regeneration

```
# Documentation tooling is optional; it is not an application runtime dependency.
python3 -m venv .venv-docs
# Activate the virtual environment for your operating system, then:
python -m pip install -r docs/requirements.txt
python tools/generate_program_documentation.py
```

The generator reads source, test XML and synthetic screenshot assets. It writes LifeSync_Program_Documentation.pdf, docs/requirements_traceability.json and docs/verification_summary.json. It does not read .env or actual account JSON. Re-run tests first; refresh screenshots after visible UI changes. Render and inspect the PDF before handing it off.

13.4 Maintenance checklist

For a future change: identify the affected requirement; preserve current user records; implement at the owning service/controller; add a regression case when behavior changes; run the full suite; inspect both themes; check late-night and nested schedules when time logic changes; update diagrams/algorithms and regenerate this manual. Avoid declaring a feature complete solely because a placeholder control exists.

14. Limitations, glossary and source index

14.1 Explicit limitations

Google Calendar is excluded. There is no external calendar synchronization, OAuth flow or calendar notification service. The application is a local desktop prototype with JSON persistence, not a concurrent server or clinical system. Recipes/favorite flags and theme selection are installation-wide; other primary records are scoped by userId. Chat and login session state do not survive a restart. Persistent Remember me behavior is not implemented. Recipe matching is lexical and does not prove stock quantities, ingredient safety or nutrient accuracy. AI output is provider-dependent and may be replaced by local fallback. Per-account favorites, encrypted storage, stronger password KDFs and platform installers would require separate scoped work.

14.2 Glossary

Term	Meaning in this program
Activity	A dated local scheduled interval: workout, meal, study, rest or other.
BMI / BMR / TDEE	Body Mass Index / estimated Basal Metabolic Rate / estimated Total Daily Energy Expenditure as computed by UserService.
CRUD	Create, read, update and delete record operations.
FXML	Declarative JavaFX view markup loaded by FXMLLoader.
Gson / JSON	The serializer and structured text representation used for persistence.
Half-open interval	An interval including its start but excluding its end; adjacent slots do not overlap.
Matrix	A two-dimensional double array; daily food entries are rows and four nutrient metrics are columns.
Recovery code	A saved high-entropy one-time secret used to authorize a local password reset.
Scope guard / generation	Topic checking and an in-memory version used to ignore obsolete chat responses.
SDLC	The planning-through-maintenance development process summarized in the supplied screenshot.

14.3 Implementation reference index

Area	Primary repository source
Startup/navigation	Launcher.java; LifeSyncApplication.java; SceneManager.java
Authentication/recovery	service/AuthService.java; controller/LoginController.java; controller/AccountRecoveryDialogs.java
Profile calculations	service/UserService.java; model/User.java; controller/ProfileController.java
Nutrition matrix	service/NutritionService.java; model/NutritionEntry.java; controller/HomeController.java
Scheduling/hierarchy	service/ActivityService.java; model/Activity.java; model/ActivityTime.java; controller/PlanController.java
Pantry and recipes	service/PantryService.java; service/RecipeService.java; controller/PantryController.java; controller/MealsController.java
AI and conversation	service/GeminiService.java; service/ChatScopeValidator.java; controller/ChatController.java
Persistence/theme	service/DataStore.java; service/ThemeManager.java; resources/com/lifesync/lifesync/css/
Verification	src/test/java/; target/surefire-reports/; docs/verification_summary.json
Proposal evidence	Unknown.jpeg; Unknown-2.jpeg; docs/requirements_traceability.json

14.4 Handoff summary

The screenshot programming requirements are mapped to code, the explicit nutrition matrix and generated-favorite persistence gaps are implemented, and the current Maven suite reports 75 passing tests with zero failures/errors. This PDF, its generator, requirement traceability and verification summary form the documentation handoff. Google Calendar remains outside the implementation scope.